



南開大學
Nankai University

《并行与分布式程序设计》实验报告

(2025~2026 学年第一学期)

实验名称: SSE 编程练习

姓 名: 李宗杭

学 号: 2311451

专 业: 软件工程

2025 年 11 月 4 日

目 录

1	矩阵乘法的优化	2
1.1	问题描述	2
1.2	算法设计与实现	2
1.2.1	计时策略	2
1.2.2	串行算法	2
1.2.3	Cache 优化	2
1.2.4	SSE 编程	3
1.2.5	分片策略	4
1.3	实验及结果分析	5
1.3.1	测试方法	5
1.3.2	四种策略下运行时间的分析	6
1.3.3	分片大小的影响分析	7
1.3.4	实际运行结果	9
2	高斯消元法 SSE/AVX 并行化	10
2.1	问题描述	10
2.2	算法设计与实现	10
2.2.1	高斯消元法	10
2.2.2	计时策略	10
2.2.3	串行算法	11
2.2.4	SSE 编程	12
2.2.5	AVX 编程	13
2.3	实验及结果分析	14
2.3.1	测试方法	14
2.3.2	实验结果分析	16
2.3.3	实际运行结果	19

1 矩阵乘法的优化

1.1 问题描述

采用串行算法、Cache 优化算法、SSE 编程和分片策略四种方式实现矩阵乘法运算，并分析比较在不同策略算法下矩阵乘法运算的时间差异。自定义矩阵大小，针对不同矩阵规模随机生成矩阵元素值。重复多次计算过程，计算平均时间来实现计时精度。

1.2 算法设计与实现

1.2.1 计时策略

即使是精度最高的计时机制，也可能测量不出太快的计算过程。于是对于每个矩阵规模、每种算法，都采用连续运算 20 次取平均的测量方法。为保证随机性，每次测量均重新随机生成矩阵元素值。此方法可同时降低误差。

```
1  double measure_time(void (*mul_func)(int), int n, int repeat) {
2      double start = get_time();
3      for (int t = 0; t < repeat; ++t) {
4          mul_func(n);
5      }
6      return (get_time() - start) / repeat; // 返回平均时间
7  }
```

1.2.2 串行算法

设计思路：采用三重嵌套循环遍历矩阵元素，按矩阵乘法定义逐元素计算结果矩阵 $C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$ 。

复杂性分析：时间复杂度为 $O(n^3)$ ，三重循环需执行 n^3 次乘法和 n^3 次加法运算；空间复杂度为 $O(n^2)$ ，仅需存储输入输出三个矩阵。

```
1  void serial_mul(int n) {
2      for (int i = 0; i < n; ++i) {
3          for (int j = 0; j < n; ++j) {
4              for (int k = 0; k < n; ++k) {
5                  c[i][j] += a[i][k] * b[k][j];
6              }
7          }
8      }
9  }
```

1.2.3 Cache 优化

设计思路：利用矩阵按行存储的内存布局特性，对矩阵 B 进行转置，将原算法中对 B 矩阵的列访问转化为行访问，提升缓存命中率。

复杂性分析：时间复杂度仍为 $O(n^3)$ ，但增加了 $O(n^2)$ 的转置操作开销；空间复杂度保持 $O(n^2)$ ，缓存命中率提升减少了内存访问延迟，实际运行效率显著提高。

```

1 void cache_mul(int n) {
2     // 转置矩阵 b，将列访问转为行访问
3     for (int i = 0; i < n; ++i) {
4         for (int j = 0; j < n; ++j) {
5             b_t[i][j] = b[j][i];
6         }
7     }
8     // 行访问优化
9     for (int i = 0; i < n; ++i) {
10        for (int j = 0; j < n; ++j) {
11            c[i][j] = 0.0f;
12            for (int k = 0; k < n; ++k) {
13                c[i][j] += a[i][k] * b_t[j][k];
14            }
15        }
16    }
17 }

```

1.2.4 SSE 编程

设计思路：基于 SIMD 指令集的单指令多数据流特性，SSE 版本使用 128 位 XMM 寄存器，每次并行处理 4 个单精度浮点数，通过向量加载、乘法、加法指令批量计算，最后处理剩余不足向量长度的元素。

复杂性分析：时间复杂度理论上降至 $O\left(\frac{n^3}{m}\right)$ (m 为每次并行处理的元素数，SSE 中 $m=4$)；空间复杂度不变，但向量指令减少了循环迭代次数和指令执行开销，同时需额外存储向量寄存器数据。

```

1 void sse_mul(int n) {
2     for (int i = 0; i < n; ++i) {
3         for (int j = 0; j < n; ++j) {
4             b_t[i][j] = b[j][i];
5         }
6     }
7     __m128 vec_a, vec_b, vec_sum;
8     for (int i = 0; i < n; ++i) {
9         for (int j = 0; j < n; ++j) {
10            c[i][j] = 0.0f;
11            vec_sum = _mm_setzero_ps();
12            // 批量处理 4 个元素
13            for (int k = n - 4; k > 0; k -= 4) {
14                vec_a = _mm_loadu_ps(a[i] + k);

```

```

15         vec_b = _mm_loadu_ps(b_t[j] + k);
16         vec_a = _mm_mul_ps(vec_a, vec_b);
17         vec_sum = _mm_add_ps(vec_sum, vec_a);
18     }
19     // 水平累加结果
20     vec_sum = _mm_hadd_ps(vec_sum, vec_sum);
21     vec_sum = _mm_hadd_ps(vec_sum, vec_sum);
22     _mm_store_ss(&c[i][j], vec_sum);
23     // 处理剩余元素
24     for (int k = n % 4 - 1; k > 0; k--) {
25         c[i][j] += a[i][k] * b_t[j][k];
26     }
27 }
28 }
29 }

```

1.2.5 分片策略

设计思路：将大矩阵划分为尺寸为 $T \times T$ 的子矩阵，通过分块计算减少缓存失效，使子矩阵数据在计算过程中持续驻留缓存，结合 SSE 向量指令进一步提升性能。

复杂性分析：时间复杂度仍为 $O(n^3)$ ，但分块操作使矩阵元素的缓存访问次数从 $O(n)$ 降至 $O(\frac{n}{T})$ (T 为子矩阵尺寸)；空间复杂度不变，缓存局部性的优化显著降低了内存访问延迟。

```

1 void tile_sse_mul(int n, int T) {
2     for (int i = 0; i < n; ++i) {
3         for (int j = 0; j < n; ++j) {
4             b_t[i][j] = b[j][i];
5         }
6     }
7     __m128 vec_a, vec_b, vec_sum;
8     float tmp;
9     // 按分片大小遍历
10    for (int r = 0; r < n / T; r++) {
11        for (int q = 0; q < n / T; q++) {
12            // 初始化当前子矩阵
13            for (int i = 0; i < T; ++i) {
14                for (int j = 0; j < T; ++j) {
15                    c[r * T + i][q * T + j] = 0.0f;
16                }
17            }
18            // 处理子矩阵
19            for (int p = 0; p < n / T; p++) {

```

```

20         for (int i = 0; i < T; ++i) {
21             for (int j = 0; j < T; ++j) {
22                 vec_sum = _mm_setzero_ps();
23                 for (int k = 0; k < T; k += 4) {
24                     vec_a = _mm_loadu_ps(&a[r*T + i][p*T + k]);
25                     vec_b = _mm_loadu_ps(&b_t[q*T + j][p*T + k]);
26                     vec_a = _mm_mul_ps(vec_a, vec_b);
27                     vec_sum = _mm_add_ps(vec_sum, vec_a);
28                 }
29                 // 累加结果
30                 vec_sum = _mm_hadd_ps(vec_sum, vec_sum);
31                 vec_sum = _mm_hadd_ps(vec_sum, vec_sum);
32                 _mm_store_ss(&tmp, vec_sum);
33                 c[r*T + i][q*T + j] += tmp;
34             }
35         }
36     }
37 }
38 }
39 }

```

1.3 实验及结果分析

1.3.1 测试方法

针对从 128 维到 2048 维共 16 种矩阵规模（每次增长 128 维），每种规模都分别使用 4 种策略进行测试，每次测试连续计算二十轮，取平均时间，每次测试重新随机初始化矩阵元素值；同时对于每种规模分别测试从 8 到 256 共 6 种分片大小的运算时间，计时方式同前。

其中“原总时长”为每行测试实际用时（20 轮），用来监测程序是否正常运行。

```

1 // 测试函数
2 void test_mul() {
3     const int REPEAT = 20;
4
5     printf("%-10s %-10s %-10s %-10s %-10s %-10s\n",
6         " 规模", " 串行算法", "Cache 优化", "SSE 版本", " 分片 SSE", " 原总时
7         ↳ 长");
8
9     for (int i = 128; i <= 2048; i += 128){
10         init_matrix(i);
11
12         double t_serial = measure_time(serial_mul, i, REPEAT);
13         double t_cache = measure_time(cache_mul, i, REPEAT);

```

```

13         double t_sse = measure_time(sse_mul, i, REPEAT);
14         double t_tile = measure_time(tile_sse_mul, i, 64, REPEAT);
15         double total = (t_serial + t_cache + t_sse + t_tile) * 20;
16
17         printf("%-10d %-10.5g %-10.5g %-10.5g %-10.5g %-10.5g\n",
18             i, t_serial, t_cache, t_sse, t_tile, total);
19     }
20
21     printf("\n%-10s %-10s %-10s %-10s %-10s %-10s %-10s %-10s\n",
22         " 规模", "8", "16", "32", "64", "128", "256", " 原总时长");
23
24     for (int i = 256; i <= 2048; i *= 2) {
25         init_matrix(i);
26         printf("%-10d", i);
27         double total = 0;
28         for (int j = 8; j <= 256; j *= 2) {
29             double avg_time = measure_time(tile_sse_mul, i, j, REPEAT);
30             printf(" %-10.5g", avg_time);
31             total += avg_time;
32         }
33         printf(" %-10.5g\n", total * 20);
34     }
35     printf(" 测试结束");
36 }

```

1.3.2 四种策略下运行时间的分析

通过图1.1可看出，所有算法策略的运行时间均随数据规模增长而延长，串行算法所耗费的时间最多，基于串行算法分析其他策略。Cache 优化算法将原算法中对矩阵的列访问转化为行访问，提升缓存命中率，因此耗时间比串行时间略少，但提升幅度在矩阵规模较小时并不是很明显，矩阵规模为 768 时逐渐显现。SSE 编程在 Cache 优化的基础上采用了并行计算，极大减少了运行时间，与串行和仅 Cache 优化的运行时间差距显著，且在 512 维时就初见端倪。而分片策略尽管理论上能提高运行速度，但实际上运行时间比 SSE 编程的略高，推测是分片逻辑引入了少量额外开销，但仍远优于 Cache 优化。

此次实验，在四种矩阵乘法方法中，性能比较：SSE 编程 > 分片策略 > Cache 优化 > 串行算法。

另外，随着数据规模增大，各优化策略的“时间节省幅度”也随之扩大，即大规模场景下，优化策略的实际价值更高。小规模（128）：SSE 版本（0.0013527）比串行算法（0.0048693）仅节省 0.0035；大规模（2048）：SSE 版本（5.9265）比串行算法（29.09）节省 23.16，节省幅度扩大超 6600 倍。

规模	串行算法	Cache 优化	SSE 版本	分片 SSE	原总时长
128	0.0048693	0.003406	0.0013527	0.0017839	0.22824
256	0.036231	0.026706	0.010967	0.014913	1.7763
384	0.14481	0.089751	0.037904	0.048109	6.4115
512	0.35675	0.21077	0.089256	0.11446	15.425
640	0.70395	0.41193	0.17434	0.21892	30.183
768	1.2349	0.71085	0.30167	0.37679	52.484
896	1.9695	1.1264	0.48341	0.59922	83.572
1024	3.0247	1.703	0.72876	0.91188	127.37
1152	4.3703	2.4123	1.032	1.291	182.11
1280	5.9216	3.3096	1.4166	1.7537	248.03
1408	8.2346	4.407	1.9078	2.3384	337.75
1536	10.768	5.7161	2.454	3.0274	439.3
1664	14.062	7.2944	3.1507	3.8502	567.14
1792	18.216	9.0616	3.9114	4.809	719.95
1920	22.972	11.128	4.8082	5.9056	896.28
2048	29.09	13.52	5.9265	7.301	1116.8

表 1: 四种策略下运行时间统计表 (单位: s)

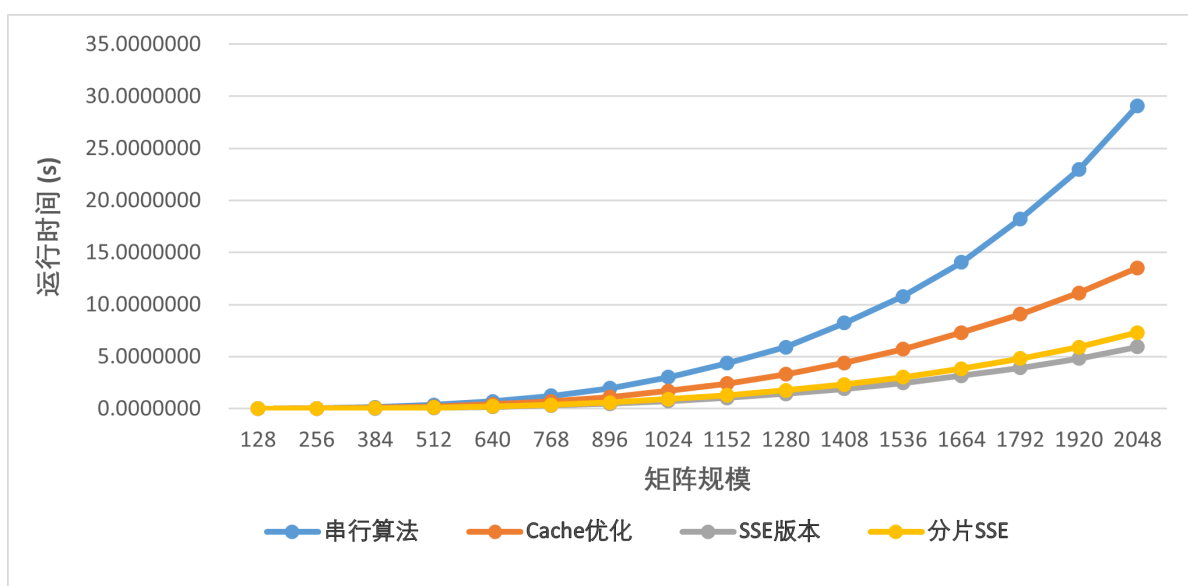


图 1.1: 四种策略下运行时间的折线图

1.3.3 分片大小的影响分析

实验中所测试的最大分块大小为 256 维, 我的 L1 为 2.1MB, $3 \times 256 \times 256 \times 4B = 786432B = 0.75MB$, 所以三个分块子矩阵均能装入 L1。

通过图1.2可看出, 对任意固定的任务规模 (256/512/1024/2048), 运行时间均随分片大小 (8→256) 增大而持续减少, 即分片越大, 效率越高。推测原因为更大的分片能减少“分片间切换”的额外开销。另外, 当任务规模越大时, 分片优化效果越显著。

因此, 此次实验得出的结果是, 分片大小越大, 运行时间越短; 且任务规模越大, 分片大小的优化

效果越显著。

规模	8	16	32	64	128	256	原总时长
256	0.02324	0.018885	0.016528	0.01438	0.013116	0.012156	1.9661
512	0.17512	0.1476	0.12929	0.11532	0.1032	0.096961	15.35
1024	1.4232	1.1335	1.008	0.90565	0.80742	0.76584	120.87
2048	11.167	9.0373	8.0007	7.1883	6.3729	6.1047	957.43

表 2: 不同分片大小下运行时间统计表 (单位: s)

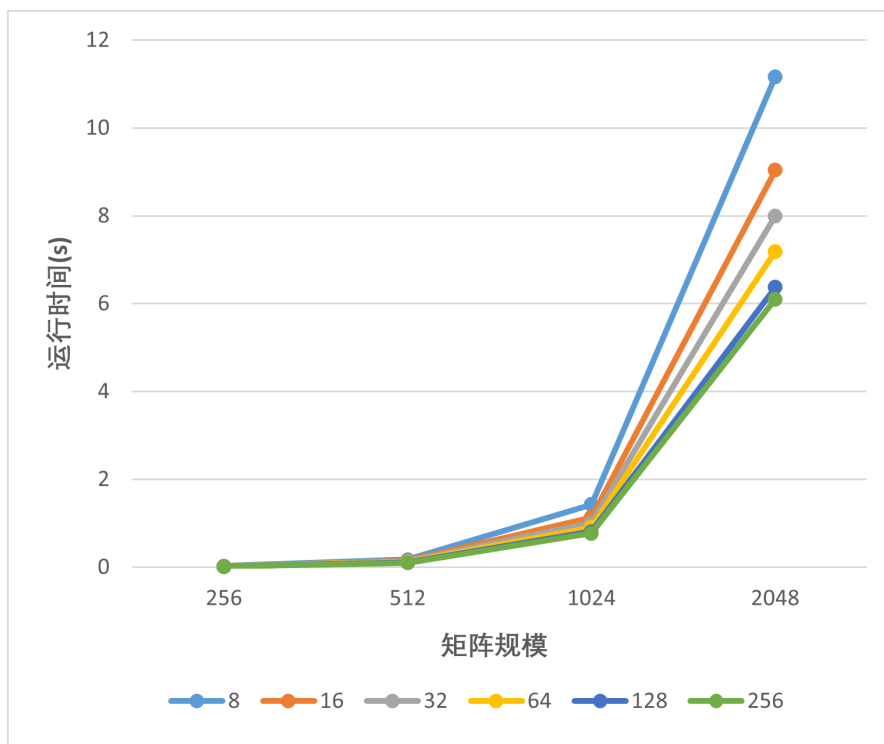
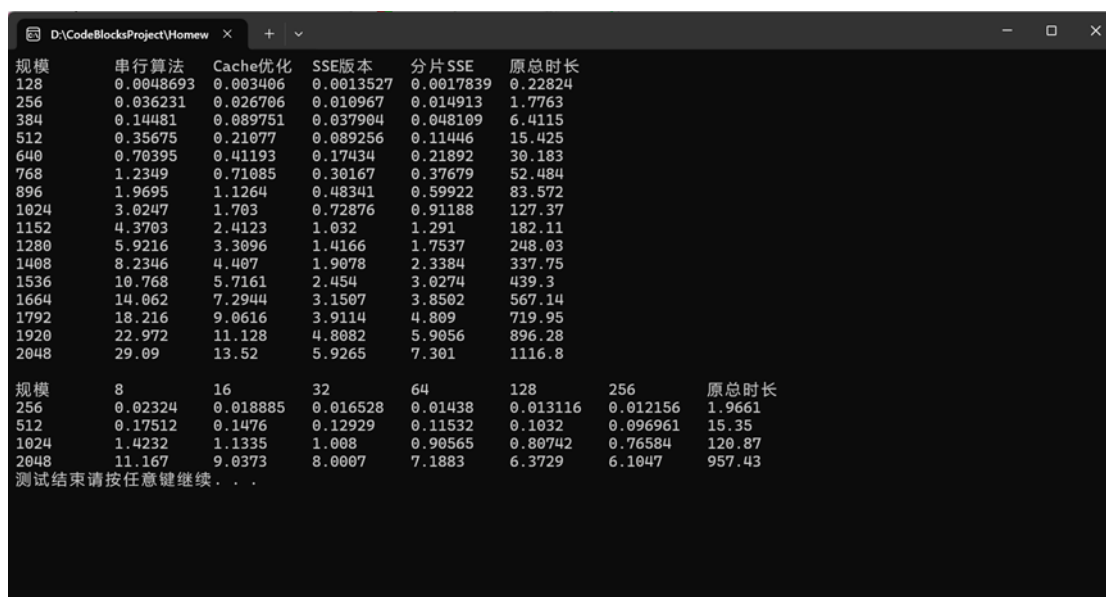


图 1.2: 四种策略下运行时间的折线图

1.3.4 实际运行结果



规模	串行算法	Cache优化	SSE版本	分片SSE	原总时长
128	0.0048693	0.003406	0.0013527	0.0017839	0.22824
256	0.036231	0.026706	0.010967	0.014913	1.7763
384	0.14481	0.089751	0.037904	0.048109	6.4115
512	0.35675	0.21077	0.089256	0.11446	15.425
640	0.70395	0.41193	0.17434	0.21892	30.183
768	1.2349	0.71085	0.30167	0.37679	52.484
896	1.9695	1.1264	0.48341	0.59922	83.572
1024	3.0247	1.703	0.72876	0.91188	127.37
1152	4.3703	2.4123	1.032	1.291	182.11
1280	5.9216	3.3096	1.4166	1.7537	248.03
1408	8.2346	4.407	1.9078	2.3384	337.75
1536	10.768	5.7161	2.454	3.0274	439.3
1664	14.062	7.2944	3.1507	3.8502	567.14
1792	18.216	9.0616	3.9114	4.809	719.95
1920	22.972	11.128	4.8082	5.9056	896.28
2048	29.09	13.52	5.9265	7.301	1116.8

规模	8	16	32	64	128	256	原总时长
256	0.02324	0.018885	0.016528	0.01438	0.013116	0.012156	1.9661
512	0.17512	0.1476	0.12929	0.11532	0.1032	0.096961	15.35
1024	1.4232	1.1335	1.008	0.90565	0.80742	0.76584	120.87
2048	11.167	9.0373	8.0007	7.1883	6.3729	6.1047	957.43

测试结束请按任意键继续...

图 1.3: 矩阵乘法测试控制台输出

2 高斯消元法 SSE/AVX 并行化

2.1 问题描述

熟悉高斯消元法解线性方程组的过程，然后实现 SSE 算法编程。过程中，请自行构造合适的线性方程组，并从以下 5 个角度，讨论不同算法策略对性能的影响。

- (1) 相同算法对于不同问题规模的性能提升是否有影响，影响情况如何；
- (2) 消元过程中采用向量编程的性能提升情况如何；
- (3) 回代过程可否向量化，有的话性能提升情况如何；
- (4) 数据对齐与不对齐对计算性能有怎样的影响；
- (5) SSE 编程和 AVX 编程性能对比。

本实验对 (1) (2) (3) (5) 四个问题进行探讨

2.2 算法设计与实现

2.2.1 高斯消元法

(1) **消元过程**：将 $Ax=b$ 按照从上至下、从左至右的顺序化为上三角方程组，中间过程不对矩阵进行交换，主要步骤如下

Step1: 将第 2 行至第 n 行，每行分别与第一行做运算，消掉每行第一个参数。

Step2: 从新矩阵的 a_{22} 开始 (a_{22} 不能为 0)，以第二行为基准，将第三行至第 n 行分别与第二行做运算，消掉每行第二个参数。

Step K: 按照上述方法，进行第 k 步运算。

Step $n-1$: 经过 $n-1$ 步，方程组也就转化为了希望得到的上三角方程组。

(2) **回代过程**：再从第 n 行开始，倒序回代前面的行中，即可求解。

2.2.2 计时策略

与矩阵乘法一样，这里采取测量 20 次取平均的策略，但不同的是这里无法直接连续运算 20 次再取平均，因为每次运算包括消元和回代过程，而我们要分别获取两个过程的时间，所以此处时间测量方式略有变化。

```
1 // 计算时间（非连续多次测量取平均）
2 pair<double, double> measure_time_gauss(void (*f_elimination)(int), void
   ↳ (*f_back)(int), int n, int repeat) {
3     double start, middle, finish;
4     double t_elimination = 0, t_back = 0;
5     for (int t = 0; t < repeat; ++t) {
6         init_gauss(n);
7         start = get_time();
8         f_elimination(n);
9         middle = get_time();
10        f_back(n);
11        finish = get_time();
12        t_elimination += middle - start;
```

```

13         t_back += finish - middle;
14     }
15     return {t_elimination / repeat, t_back / repeat}; // 返回平均时间
16 }

```

2.2.3 串行算法

(1) 消元过程:

对于系数矩阵 A, 从第一行开始, 遍历每一行, 作为被减行 k, 在每轮循环中, 遍历被减行下面的每一行, 对于每一行 i, 遍历其从第 k 列开始的所有列 j, 将所有元素 A[i][j] 减去该行第 k 列元素与被减行的第 k 列元素之比与被减行该列元素的乘积, 同时常数项该行元素也要做类似减法。

(2) 回代过程:

第 n 行为一元一次方程, 直接求得 X[n-1], 随后不断用已求得的 X 求解上一行

复杂性分析:

$$\sum_{k=1}^{n-1} (n-k+1)^2 = \frac{n(n+1)(2n+1)}{6} - 1$$

$$\sum_{i=n}^1 (n-i+1) = \frac{n(n+1)}{2}$$

$$\frac{n^3}{3} + n^2 + \frac{2n}{3} - 1 \approx \frac{n^3}{3} = O(n^3)$$

其中消元过程 $O(n^3)$, 回代过程 $O(n^2)$

简而言之, 串行算法的核心在于对于每行, 要逐个处理每列元素。

```

1 // 串行高斯消元
2 void serial_elimination(int n) {
3     for (int k = 0; k < n - 1; ++k) {
4         for (int i = k + 1; i < n; ++i) {
5             float factor = A[i][k] / A[k][k];
6             for (int j = k; j < n; ++j) {
7                 A[i][j] -= factor * A[k][j];
8             }
9             B[i] -= factor * B[k];
10        }
11    }
12 }
13
14 // 串行回代
15 void serial_back(int n) {
16     X[n - 1] = B[n - 1] / A[n - 1][n - 1];
17     for (int i = n - 2; i >= 0; --i) {
18         float sum = B[i];

```

```

19         for (int j = i + 1; j < n; ++j) {
20             sum -= A[i][j] * X[j];
21         }
22         X[i] = sum / A[i][i];
23     }
24 }

```

2.2.4 SSE 编程

SSE/AVX 并行化编程与串行的唯一区别就是对于每行元素的处理，由一个个处理变为多个同时处理

复杂性分析：

SSE/AVX 并行化后，消元过程时间复杂度降至 $O\left(\frac{n^3}{m}\right)$ ，回代过程降至 $O\left(\frac{n^2}{m}\right)$ ，(m 为每次并行处理的元素数，SSE 中 m=4，AVX 中 m=8)

```

1  // SSE 高斯消元
2  void sse_elimination(int n) {
3      for (int k = 0; k < n - 1; ++k) {
4          for (int i = k + 1; i < n; ++i) {
5              float factor = A[i][k] / A[k][k];
6              int j = k;
7              __m128 vec_factor = _mm_set1_ps(factor);
8              for (; j <= n - 4; j += 4) {
9                  __m128 vec_A_k = _mm_loadu_ps(A[k] + j);
10                 __m128 vec_A_i = _mm_loadu_ps(A[i] + j);
11                 vec_A_i = _mm_sub_ps(vec_A_i, _mm_mul_ps(vec_factor, vec_A_k));
12                 _mm_storeu_ps(A[i] + j, vec_A_i);
13             }
14             // 剩余元素
15             for (; j < n; ++j) {
16                 A[i][j] -= factor * A[k][j];
17             }
18             B[i] -= factor * B[k];
19         }
20     }
21 }
22
23 // SSE 回代
24 void sse_back(int n) {
25     X[n - 1] = B[n - 1] / A[n - 1][n - 1];
26     for (int i = n - 2; i >= 0; --i) {
27         float sum = B[i];

```

```

28     __m128 vec_sum = _mm_set1_ps(sum);
29     int j = i + 1;
30     for (; j <= n - 4; j += 4) {
31         __m128 vec_A_i = _mm_loadu_ps(A[i] + j);
32         __m128 vec_x_j = _mm_loadu_ps(X + j);
33         vec_sum = _mm_sub_ps(vec_sum, _mm_mul_ps(vec_A_i, vec_x_j));
34     }
35     // 累加结果
36     vec_sum = _mm_hadd_ps(vec_sum, vec_sum);
37     vec_sum = _mm_hadd_ps(vec_sum, vec_sum);
38     _mm_storeu_ps(&sum, vec_sum);
39     // 剩余元素
40     for (; j < n; ++j) {
41         sum -= A[i][j] * X[j];
42     }
43     X[i] = sum / A[i][i];
44 }
45 }

```

2.2.5 AVX 编程

```

1  // AVX 高斯消元
2  void avx_elimination(int n) {
3      for (int k = 0; k < n - 1; ++k) {
4          for (int i = k + 1; i < n; ++i) {
5              float factor = A[i][k] / A[k][k];
6              int j = k;
7              __m256 vec_factor = _mm256_set1_ps(factor);
8              for (; j <= n - 8; j += 8) {
9                  __m256 vec_A_k = _mm256_loadu_ps(A[k] + j);
10                 __m256 vec_A_i = _mm256_loadu_ps(A[i] + j);
11                 vec_A_i = _mm256_sub_ps(vec_A_i, _mm256_mul_ps(vec_factor,
12                     ↪ vec_A_k));
13                 _mm256_storeu_ps(A[i] + j, vec_A_i);
14             }
15             // 处理剩余元素
16             for (; j < n; ++j) {
17                 A[i][j] -= factor * A[k][j];
18             }
19             B[i] -= factor * B[k];
20     }
21 }

```

```
21 }
22
23 // AVX 回代
24 void avx_back(int n) {
25     X[n - 1] = B[n - 1] / A[n - 1][n - 1];
26     for (int i = n - 2; i >= 0; --i) {
27         float sum = B[i];
28         __m256 vec_sum = _mm256_set1_ps(sum);
29         int j = i + 1;
30         for (; j <= n - 8; j += 8) {
31             __m256 vec_A_i = _mm256_loadu_ps(A[i] + j);
32             __m256 vec_x_j = _mm256_loadu_ps(X + j);
33             vec_sum = _mm256_sub_ps(vec_sum, _mm256_mul_ps(vec_A_i, vec_x_j));
34         }
35         // 累加向量结果
36         __m128 s1 = _mm256_extractf128_ps(vec_sum, 0);
37         __m128 s2 = _mm256_extractf128_ps(vec_sum, 1);
38         s1 = _mm_hadd_ps(s1, s2);
39         s1 = _mm_hadd_ps(s1, s1);
40         _mm_storeu_ps(&sum, s1);
41         // 处理剩余元素
42         for (; j < n; ++j) {
43             sum -= A[i][j] * X[j];
44         }
45         X[i] = sum / A[i][i];
46     }
47 }
```

2.3 实验及结果分析

2.3.1 测试方法

针对从 128 维到 2048 维共 16 种矩阵规模（每次增长 128 维），每种规模都分别使用 3 种策略进行测试，每次测试计算二十轮，取平均时间，每次测试重新随机初始化矩阵元素值。

```
1 // 测试函数
2 void test_gauss() {
3     const int REPEAT = 20;
4     double back_result[16][6];
5     pair<double, double> result;
6
7     printf(" 消元\n");
8     printf("%-10s %-10s %-10s %-10s %-10s %-10s\n",
```

```

9      " 规模", " 串行", "SSE", "AVX", "SSE 提升", "AVX 提升");
10  int k = 0;
11  for (int i = 128; i <= 2048; i += 128){
12      back_result[k][0] = i;
13      // 串行
14      result = measure_time_gauss(serial_elimination, serial_back, i, REPEAT);
15      double t_serial_elimination = result.first;
16      back_result[k][1] = result.second;
17      // SSE
18      result = measure_time_gauss(sse_elimination, sse_back, i, REPEAT);
19      double t_sse_elimination = result.first;
20      back_result[k][2] = result.second;
21      // AVX
22      result = measure_time_gauss(avx_elimination, avx_back, i, REPEAT);
23      double t_avx_elimination = result.first;
24      back_result[k][3] = result.second;
25      // 消元效率提升百分比
26      double sse_improve = (t_serial_elimination - t_sse_elimination) /
27      ↪ t_serial_elimination * 100;
28      double avx_improve = (t_serial_elimination - t_avx_elimination) /
29      ↪ t_serial_elimination * 100;
30      // 回代效率提升百分比
31      back_result[k][4] = (back_result[k][1] - back_result[k][2]) /
32      ↪ back_result[k][1] * 100;
33      back_result[k][5] = (back_result[k][1] - back_result[k][3]) /
34      ↪ back_result[k][1] * 100;
35      k++;
36      char sse_str[20], avx_str[20];
37      sprintf(sse_str, "%.2f%%", sse_improve);
38      sprintf(avx_str, "%.2f%%", avx_improve);
39      printf("%-10d %-10.5g %-10.5g %-10.5g %-10s %-10s\n",
40      ↪ i, t_serial_elimination, t_sse_elimination, t_avx_elimination,
41      ↪ sse_str, avx_str);
42  }
43
44  printf("\n回代\n");
45  printf("%-10s %-10s %-10s %-10s %-10s %-10s\n",
46      " 规模", " 串行", "SSE", "AVX", "SSE 提升", "AVX 提升");
47
48  for (int i = 0; i < 16; i++) {
49      char sse_back_str[20], avx_back_str[20];
50      sprintf(sse_back_str, "%.2f%%", back_result[i][4]);

```



```

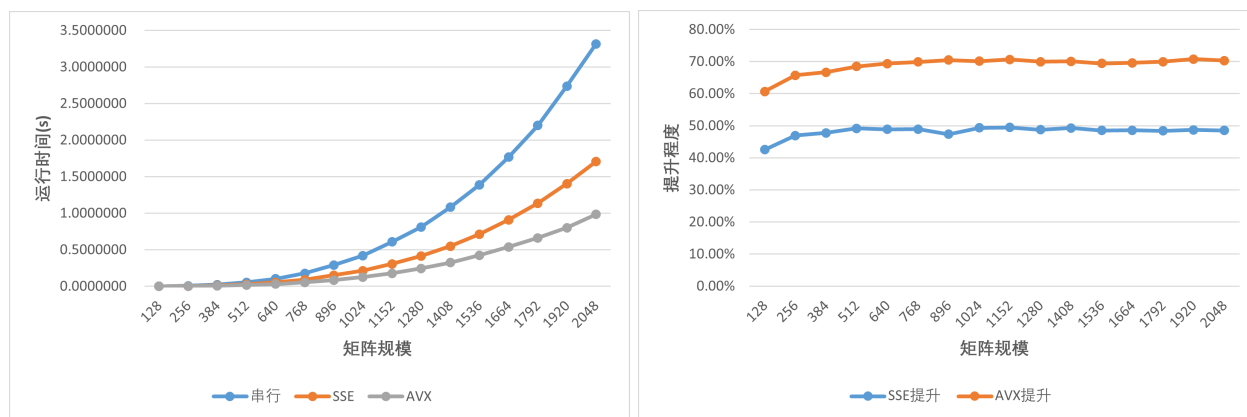
46     sprintf(avx_back_str, "%.2f%%", back_result[i][5]);
47     printf("%-10d %-10.5g %-10.5g %-10.5g %-10s %-10s\n",
48           (int)back_result[i][0], back_result[i][1], back_result[i][2],
49           ↪ back_result[i][3], sse_back_str, avx_back_str);
50 }
51 printf(" 测试结束");
52 }

```

2.3.2 实验结果分析

规模	串行	SSE	AVX	SSE 提升	AVX 提升
128	0.00090132	0.00051805	0.00035426	42.52%	60.70%
256	0.0067122	0.00356	0.0023007	46.96%	65.72%
384	0.022329	0.011666	0.0074384	47.75%	66.69%
512	0.053538	0.027224	0.016887	49.15%	68.46%
640	0.10356	0.052955	0.031761	48.87%	69.33%
768	0.17696	0.090325	0.053273	48.96%	69.89%
896	0.29068	0.15304	0.085795	47.35%	70.48%
1024	0.41998	0.21286	0.12535	49.32%	70.15%
1152	0.6086	0.30755	0.17867	49.47%	70.64%
1280	0.8104	0.41528	0.24381	48.76%	69.92%
1408	1.0829	0.54887	0.32414	49.31%	70.07%
1536	1.3889	0.71473	0.42482	48.54%	69.41%
1664	1.7663	0.90808	0.53711	48.59%	69.59%
1792	2.2013	1.1359	0.66206	48.40%	69.92%
1920	2.7379	1.4053	0.79985	48.67%	70.79%
2048	3.3123	1.7057	0.98427	48.50%	70.28%

表 3: 消元过程不同策略运行时间及提升率统计表 (单位: s)



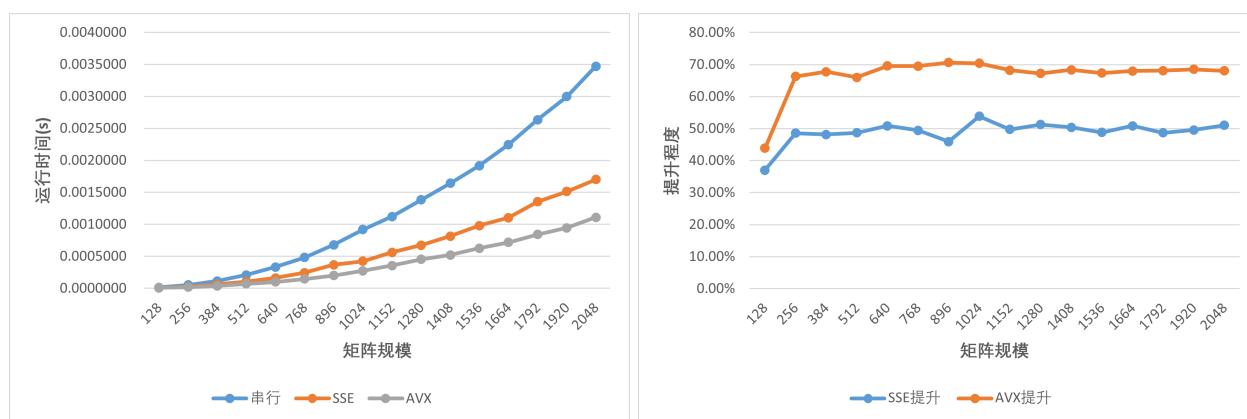
(a) 消元过程运行时间

(b) 消元过程提升程度

图 2.4: 消元过程数据图

规模	串行	SSE	AVX	SSE 提升	AVX 提升
128	0.00001342	0.000008455	0.000007535	37.00%	43.85%
256	0.000053465	0.000027515	0.000018025	48.54%	66.29%
384	0.00011585	0.00006007	0.00003738	48.15%	67.73%
512	0.00020731	0.00010638	0.000070435	48.69%	66.02%
640	0.00033101	0.00016279	0.00010063	50.82%	69.60%
768	0.00048211	0.0002437	0.00014686	49.45%	69.54%
896	0.00068078	0.00036837	0.00020004	45.89%	70.62%
1024	0.00091793	0.00042354	0.00027189	53.86%	70.38%
1152	0.0011207	0.00056306	0.00035602	49.76%	68.23%
1280	0.0013821	0.00067372	0.00045305	51.25%	67.22%
1408	0.0016435	0.0008153	0.0005207	50.39%	68.32%
1536	0.0019164	0.00098083	0.00062612	48.82%	67.33%
1664	0.0022452	0.0011041	0.00071857	50.82%	68.00%
1792	0.0026348	0.001353	0.00084048	48.65%	68.10%
1920	0.0029958	0.0015121	0.00094368	49.53%	68.50%
2048	0.0034716	0.0017002	0.0011096	51.03%	68.04%

表 4: 回代过程不同策略运行时间及提升率统计表 (单位: s)



(a) 回代过程运行时间

(b) 回代过程提升程度

图 2.5: 回代过程数据图

(1) 相同算法对于不同问题规模的性能提升是否有影响，影响情况如何？

从图4(b)和5(b)中可以看出，在消元和回代的过程中，均在规模较小时，性能提升幅度会随着规模的变大而微弱增长，但消元过程增长幅度也不大，只是回代过程折现有些陡峭，初步将其判断为机器合理误差。结合表3和4可以看出，在消元和回代过程中，几乎都是在规模为 640 之前略有增长，而之后便都趋于稳定。

据此可以推测出，相同算法对于不同问题规模的性能提升影响不大。

但随后，为了验证回代过程中从 128 到 256 的陡增是否为机器波动误差，我决定再次进行测试，可结果仍然在此位置有较大涨幅，于是更改测试策略，将测试规模改为从 8 到 1024，得出了不一样的结果。

没有额外制作折线图，仅从表5和6就可清晰地看出，无论是消元还是回代过程，在规模为 256 以下，性能提升率是不断变化的，整体呈现出增长趋势，但却出现一些负数。且进行了多次实验均为此情况（整体增长，存在负数）。

于是，综合所有实验结果，得出新的结论：相同算法对于不同问题规模的性能提升有影响，在问题规模较小时影响显著，总体呈现性能提升幅度随问题规模的增大而增大，但存在并行策略不如串行策略的情况（负提升）；在问题规模达到一定程度后，性能提升从波动走向稳定，且稳定在较高水平。推测原因为小规模数据的计算量远小于“向量指令调度、数据加载”等额外开销，并行优势被抵消，甚至因指令复杂度增加导致性能下降；规模增大后，“计算密集度”提升，向量指令的并行优势（一次处理多组数据）被充分释放，额外开销占比大幅降低，提升率进入稳定区间。

规模	串行	SSE	AVX	SSE 提升	AVX 提升
8	0.00000033	0.000000285	0.00000036	13.64%	-9.09%
16	0.000002345	0.000001465	0.000001345	37.53%	42.64%
32	0.00001559	0.000008685	0.000006965	44.29%	55.32%
64	0.00012908	0.000060495	0.00005121	53.13%	60.33%
128	0.00086668	0.00045924	0.00030627	47.01%	64.66%
256	0.0067958	0.0035197	0.0022033	48.21%	67.58%
512	0.053039	0.026652	0.016527	49.75%	68.84%
1024	0.41893	0.2136	0.13572	49.01%	67.60%

表 5: 消元过程不同策略运行时间及提升率统计表（单位：s）- 小规模

规模	串行	SSE	AVX	SSE 提升	AVX 提升
8	0.000000155	0.00000014	0.00000014	9.68%	9.68%
16	0.00000026	0.00000042	0.000000355	-61.53%	-36.54%
32	0.00000091	0.000000925	0.0000009	-1.65%	1.10%
64	0.000003535	0.000002715	0.000002215	23.20%	37.34%
128	0.000013475	0.00000813	0.00000607	39.67%	54.95%
256	0.00005263	0.00002875	0.000018385	45.37%	65.07%
512	0.0002118	0.00010981	0.00006274	48.16%	70.38%
1024	0.00085903	0.00044565	0.00028636	48.12%	66.66%

表 6: 回代过程不同策略运行时间及提升率统计表（单位：s）- 小规模

(2) 消元过程中采用向量编程的性能提升情况如何？

结合问题（1）中的分析可知，消元过程中，在问题规模较小时，性能提升幅度随规模变大而变大，

规模增大到一定程度后，性能提升从波动走向稳定，且稳定在较高水平（本实验中 SSE 为 48% 左右，AVX 为 70% 左右）

(3) 回代过程可否向量化，有的话性能提升情况如何？

回代过程也存在对矩阵元素（连续存储数据）的逐行逐列处理，显然可以向量化，其性能提升幅度与消元过程相似。

(4) SSE 编程和 AVX 编程性能对比

从各个图表中可以看出，AVX 编程的性能全面领先 SSE 编程，尤其是在大规模数据时。在本次实验中，AVX 的提升率比 SSE 稳定高出 20 个百分点。

2.3.3 实际运行结果

D:\CodeBlocksProject\Homew

+

×

消元

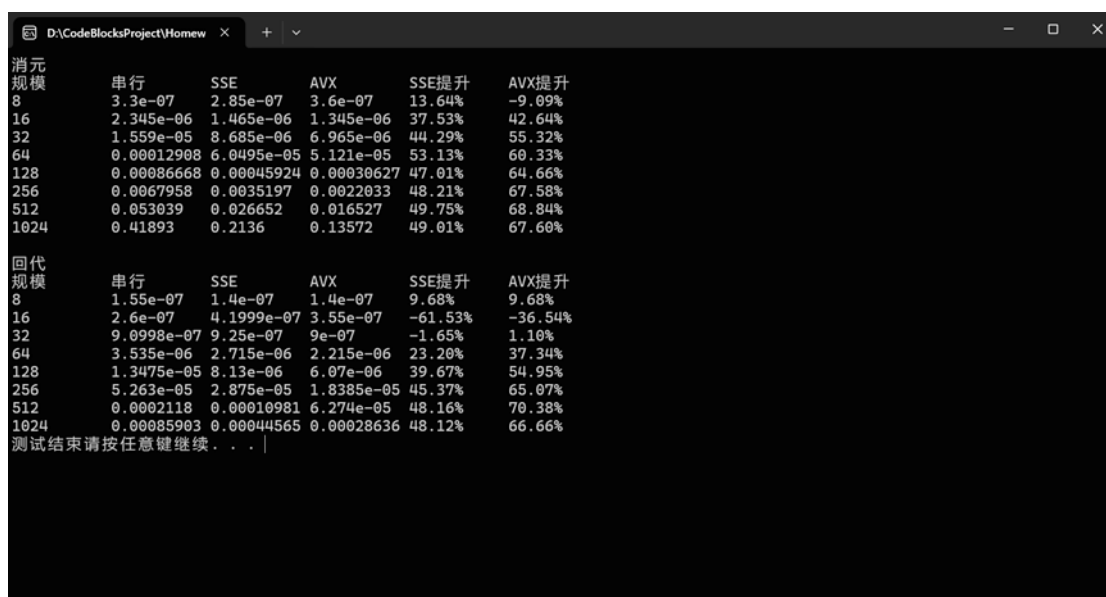
规模	串行	SSE	AVX	SSE提升	AVX提升
128	0.00090132	0.00051805	0.00035426	42.52%	60.70%
256	0.0067122	0.00356	0.0023007	46.96%	65.72%
384	0.022329	0.011666	0.0074384	47.75%	66.69%
512	0.053538	0.027224	0.016887	49.15%	68.46%
640	0.10356	0.052955	0.031761	48.87%	69.33%
768	0.17696	0.090325	0.053273	48.96%	69.89%
896	0.29068	0.15304	0.085795	47.35%	70.48%
1024	0.41998	0.21286	0.12535	49.32%	70.15%
1152	0.6086	0.30755	0.17867	49.47%	70.64%
1280	0.8104	0.41528	0.24381	48.76%	69.92%
1408	1.0829	0.54887	0.32414	49.31%	70.07%
1536	1.3889	0.71473	0.42482	48.54%	69.41%
1664	1.7663	0.90808	0.53711	48.59%	69.59%
1792	2.2013	1.1359	0.66206	48.40%	69.92%
1920	2.7379	1.4053	0.79985	48.67%	70.79%
2048	3.3123	1.7057	0.98427	48.50%	70.28%

回代

规模	串行	SSE	AVX	SSE提升	AVX提升
128	1.342e-05	8.455e-06	7.535e-06	37.00%	43.85%
256	5.3465e-05	2.7515e-05	1.8025e-05	48.54%	66.29%
384	0.00011585	6.007e-05	3.738e-05	48.15%	67.73%
512	0.00020731	0.00010638	7.0435e-05	48.69%	66.02%
640	0.00033101	0.00016279	0.00010063	50.82%	69.60%
768	0.00048211	0.0002437	0.00014686	49.45%	69.54%
896	0.00068078	0.00036837	0.00020004	45.89%	70.62%
1024	0.00091793	0.00042354	0.00027189	53.86%	70.38%
1152	0.0011207	0.00056306	0.00035602	49.76%	68.23%
1280	0.0013821	0.00067372	0.00045305	51.25%	67.22%
1408	0.0016435	0.0008153	0.0005207	50.39%	68.32%
1536	0.0019164	0.00098083	0.00062612	48.82%	67.33%
1664	0.0022452	0.0011041	0.00071857	50.82%	68.00%
1792	0.0026348	0.001353	0.00084048	48.65%	68.10%
1920	0.0029958	0.0015121	0.00094368	49.53%	68.50%
2048	0.0034716	0.0017002	0.0011096	51.03%	68.04%

测试结束请按任意键继续 . . .

图 2.6: 高斯消元法测试控制台输出



消元	串行	SSE	AVX	SSE提升	AVX提升
规模					
8	3.3e-07	2.85e-07	3.6e-07	13.64%	-9.09%
16	2.345e-06	1.465e-06	1.345e-06	37.53%	42.64%
32	1.559e-05	8.685e-06	6.965e-06	44.29%	55.32%
64	0.00012908	6.0495e-05	5.121e-05	53.13%	60.33%
128	0.00086668	0.00045924	0.00030627	47.01%	64.66%
256	0.0067958	0.0035197	0.0022033	48.21%	67.58%
512	0.053039	0.026652	0.016527	49.75%	68.84%
1024	0.41893	0.2136	0.13572	49.01%	67.60%

回代	串行	SSE	AVX	SSE提升	AVX提升
规模					
8	1.55e-07	1.4e-07	1.4e-07	9.68%	9.68%
16	2.6e-07	4.1999e-07	3.55e-07	-61.53%	-36.54%
32	9.0998e-07	9.25e-07	9e-07	-1.65%	1.10%
64	3.535e-06	2.715e-06	2.215e-06	23.20%	37.34%
128	1.3475e-05	8.13e-06	6.07e-06	39.67%	54.95%
256	5.263e-05	2.875e-05	1.8385e-05	45.37%	65.07%
512	0.0002118	0.00010981	6.274e-05	48.16%	70.38%
1024	0.00085903	0.00044565	0.00028636	48.12%	66.66%

测试结束请按任意键继续 . . . |

图 2.7: 高斯消元法测试控制台输出-小规模